Module 1: The High-Performance Data Engineer

"The Python, The Speed, The Hunter, The Analyst"

Objective: To transform you from a beginner to a developer who can write professional, high-speed code to gather and analyze data.

☑ Definition of Done (Your Checklist)

Do not move to Module 2 until you can tick these off:

1. ☐ I can write a Python Class that inherits from another Class.
2. ☐ I can explain the difference between `synchronous` (one by one) and `asynchronous` (all at once) code.
3. ☐ I have written a script using `async` and `await` that runs faster than a normal loop.
4. ☐ I can scrape data from a website that uses JavaScript (like infinite scroll).
5. ☐ I can load a JSON file into Pandas, clean the empty rows, and save it as a CSV.
6. ☐ I use Git to save my work and I write tests using `pytest`.

Phase 1.1: Professional Python Core

The Foundation. Writing code that is clean, reusable, and readable.

1.1.1 Advanced Object-Oriented Programming (OOP)

- **Simple Explanation:** Instead of writing one long script, you create "Blueprints" (Classes). For example, you make a Blueprint for a `Scraper`. Then you can create many "Scraper Robots" (Objects) from that blueprint without rewriting code.
- **Why for you?** You will need a `User` blueprint, a `Question` blueprint, and a `Test` blueprint.
- **Keywords to Study:**
  - `Class` vs `Object` (Instance)
  - `__init__` (The Constructor method)
  - `self` (Refers to the current object)
  - `Inheritance` (Parent Class vs Child Class)
  - `Encapsulation` (Private variables like `__password`)
  - `Polymorphism` (Method Overriding)

1.1.2 The "Pythonic" Toolkit (Advanced Features)

- **Simple Explanation:** Special tools in Python that make your code shorter and faster.
- **Why for you?**
  - **Decorators:** To automatically check if a user is logged in before letting them see a page.
  - **Generators:** To handle 50,000 MCQs one by one without your computer crashing (running out of RAM).
- **Keywords to Study:**
  - `Decorators` (using `@wrapper` syntax)
  - `args` and `kwargs` (Variable arguments)
  - `Generators` vs `Iterators`
  - `yield` keyword (The magic switch for generators)
  - `List Comprehensions` (Writing loops in one line)
  - `Context Managers` (`with open(...)`)
  - `Type Hinting` (`def func(x: int) -> str:`)

Phase 1.2: Asynchronous Programming (The Speed) ⚡

Making your code do multiple things at the same time.

1.2.1 The Concept: Sync vs Async

- **Simple Explanation:**
  - **Synchronous (Sync):** You call a friend. You wait on the line until they answer. You can't do anything else while waiting. (Slow).
  - **Asynchronous (Async):** You send a text. You put the phone down and do dishes. When they reply, you pick the phone up again. (Fast).
- **Why for you?** When scraping 100 websites, Sync code waits for Site 1 to finish before starting Site 2. Async code starts all 100 at once.
- **Keywords to Study:**
  - `Blocking` vs `Non-Blocking` I/O
  - `Concurrency` vs `Parallelism`
  - `The Event Loop` (How Python manages tasks)
  - `Coroutines`

1.2.2 The Syntax & Tools

- **Simple Explanation:** The specific words you type to tell Python "Don't wait here, go do something else."
- **Keywords to Study:**
  - `async def` (Defining an async function)

- `await` (Pausing here to let other tasks run)
- `asyncio` library
- `asyncio.run()`
- `asyncio.gather()` (Running tasks in parallel)
- `asyncio.sleep()` (Non-blocking sleep)

Phase 1.3: Web Scraping (The Hunt) 🕸️

Gathering the data. The "Freelance Money" skill.

[Image of web scraping architecture]

1.3.1 The Basics: Requests & HTML

- **Simple Explanation:** Fetching the raw code of a website and finding the specific text you want (like the Price or the Question).
- **Keywords to Study:**
  - `HTTP Methods` (GET vs POST)
  - `Status Codes` (200 OK, 404 Not Found, 403 Forbidden)
  - `User-Agent` Headers (Pretending to be a browser)
  - `HTML DOM Structure` (Tags, IDs, Classes)
  - `CSS Selectors` (Finding elements by class)
  - `BeautifulSoup4` (Parsing library)

1.3.2 High-Speed Async Scraping

- **Simple Explanation:** Using your Async skills to download 50 pages per second instead of 1 page per second.
- **Keywords to Study:**
  - `aiohttp` (Async version of requests)
  - `ClientSession`
  - `Rate Limiting` (Slowing down to avoid bans)
  - `Semaphores` (Limiting max concurrent connections)

1.3.3 Dynamic Scraping (The Heavy Artillery)

- **Simple Explanation:** Some sites use JavaScript to load data (like Instagram). Normal scrapers can't see it. You need a tool that controls a real browser (Chrome) invisibly.
- **Keywords to Study:**
  - `Selenium WebDriver` or `Playwright` (Recommended)
  - `Headless Browser` (Browser with no UI)
  - `DOM Manipulation` (Clicking buttons via code)
  - `Explicit Waits` (Waiting for an element to appear)

Phase 1.4: The Data Science Stack (The Lab)

Cleaning and Analyzing the data.

1.4.1 NumPy (The Engine)

- **Simple Explanation:** Python lists are slow for math. NumPy arrays are super fast.
- **Keywords to Study:**
  - `NumPy Array` vs `List`
  - `Vectorization` (Doing math on whole arrays at once)
  - `Broadcasting`

1.4.2 Pandas (The Excel Killer)

- **Simple Explanation:** Think of Pandas as "Programmable Excel." It lets you load data, delete bad rows, fix spelling, and calculate stats using code.
- **Why for you?** You will scrape messy data. Pandas makes it clean before you put it in your database.
- **Keywords to Study:**
  - `DataFrame` (The main table)
  - `Series` (A single column)
  - `pd.read_csv()` / `pd.read_json()`
  - `df.head()`, `df.info()`, `df.describe()`
  - `df.dropna()` (Remove empty rows)
  - `df.fillna()` (Fill empty rows)
  - `df.groupby()` (Grouping data, e.g., by Subject)
  - `df.to_csv()` (Saving data)

Phase 1.5: Engineering Habits (The Safety Net)

Tools that professionals use to avoid breaking things.

1.5.1 Git & GitHub

- **Simple Explanation:** A "Save Button" for your code history. If you break something, you can go back to how it was yesterday.
- **Keywords to Study:**
  - `git init`
  - `git add .`
  - `git commit -m "message"`
  - `git push`
  - `.gitignore` (Important! Hides secrets)

1.5.2 Testing (Pytest)

- **Simple Explanation:** Writing a small script that checks if your main script is working correctly.
- **Keywords to Study:**
  - `Unit Testing`
  - `pytest` framework
  - `assert` statement
  - `Test Coverage`

🛠️ The Capstone Project: "The Async Data Pipeline"

Goal: Build a script that uses *everything* you just learned.

Step 1: Create a Python Project with a virtual environment (`venv`). **Step 2:** Write an `Async Scraper` using `aiohttp`. * **Task:** Scrape 20 pages of a quote website (like `quotes.toscrape.com`). * **Requirement:** Use `asyncio.gather` to fetch them all at once. **Step 3:** Parse the HTML using `BeautifulSoup`. * **Task:** Extract the Quote Text, Author, and Tags. **Step 4:** Clean the data using `Pandas`. * **Task:** Load the data into a DataFrame. Remove any quotes that are too short. Count how many quotes each author has. **Step 5:** Save the result. * **Task:** Export to `quotes_analysis.csv`.

Module 2: The Backend Monolith

"The Architect, The Guard, and The Flash"

Objective: To transition from running scripts on your laptop to building a secure, scalable server that can handle thousands of users. You will master the database, build the API, lock the doors, and turbocharge the speed.

☑️ Definition of Done (Your Checklist)

Do not move to Module 3 until you can tick these off:

1. ☐ I can draw an ER Diagram showing how a `Student`, `Test`, and `Result` are connected.
2. ☐ I can write a raw SQL query using `INNER JOIN` to find all tests taken by a specific student.
3. ☐ I have a running FastAPI server where I can `POST` a question and `GET` it back.
4. ☐ I use Pydantic to stop users from sending bad data (like an invalid email).
5. ☐ I have implemented a "Login" system that gives me a JWT (Token).
6. ☐ I have set up Redis to cache my "Get All Questions" endpoint.

Phase 2.1: Master the Database (SQL First)

The Bedrock. Before you code the API, you must design the Brain.

2.1.1 Relational Design (Thinking in Tables)

- **Simple Explanation:** A database isn't just one big Excel sheet. It's many small tables linked together. Designing this correctly is 80% of the work.
- **Why for you?** You need to know: If a Student takes a Test, where do we save the score? In the Student table? The Test table? (Answer: Neither. In a third "Results" table).
- **Keywords to Study:**
  - `RDBMS` (Relational Database Management System)
  - `Primary Key` (The unique ID, e.g., `student_id`)
  - `Foreign Key` (The link to another table)
  - `Normalization` (1NF, 2NF, 3NF - Removing duplicate data)
  - `One-to-Many Relationship` (One Subject -> Many Chapters)
  - `Many-to-Many Relationship` (Students <-> Tests)

2.1.2 The SQL Language (Speaking to the DB)

- **Simple Explanation:** The language used to ask questions to the database. It is universal.
- **Why for you?** You need to ask complex questions like: "*Show me the average score of all students in Physics tests taken last week.*" Python is too slow for this. SQL is instant.
- **Keywords to Study:**
  - `DDL` (Create/Alter/Drop Table)
  - `DML` (Insert/Update/Delete Data)
  - `SELECT ... FROM ... WHERE`
  - `JOINS` (Inner, Left, Right - Combining tables)
  - `GROUP BY` & `HAVING` (Aggregating data)
  - `ORDER BY` & `LIMIT` (Pagination)

2.1.3 PostgreSQL Specifics

- **Simple Explanation:** The specific software we use. It's the industry standard for serious apps.
- **Keywords to Study:**
  - `PostgreSQL` vs `MySQL` (Why Postgres is better for complex data)
  - `JSONB` (Storing JSON data inside a SQL table - *Super useful for MCQ Options*)
  - `pgAdmin` or `DBeaver` (Tools to see your data visually)

Phase 2.2: The Server Framework (FastAPI)

The Interface. The waiter that takes orders from the user and gives them to the kitchen (DB).

2.2.1 REST API Architecture

- **Simple Explanation:** A set of rules for how computers talk to each other.
- **Why for you?** You need to build a system where the Frontend (App) asks the Backend (API) for data in a standard way.
- **Keywords to Study:**
  - `Client-Server Model`
  - `HTTP Methods` (GET=Read, POST=Create, PUT=Update, DELETE=Delete)
  - `URL Parameters` (`/questions/5`) vs `Query Parameters` (`/questions?subject=physics`)
  - `Status Codes` (201 Created, 400 Bad Request, 401 Unauthorized, 500 Server Error)

2.2.2 Pydantic (The Bouncer)

- **Simple Explanation:** A tool that checks data *before* it enters your system. If the data is wrong, Pydantic kicks it out.
- **Why for you?** If a user tries to register with `"email": "hello"`, Pydantic will auto-reply: "*Error: Invalid Email*" without you writing extra code.
- **Keywords to Study:**
  - `Pydantic Models` (Defining the shape of data)
  - `Field Validation` (e.g., `min_length=8` for passwords)
  - `Response Models` (Hiding passwords in the response)

2.2.3 Dependency Injection

- **Simple Explanation:** A cool FastAPI feature. It's like having a helper robot that automatically gives you things (like a Database Connection or a User ID) whenever you ask for them in a function.
- **Keywords to Study:**
  - `Depends()`
  - `Context Managers` (for DB sessions)

Phase 2.3: The Bridge (SQLAlchemy ORM)

Connecting Python to SQL without writing raw SQL every time.

2.3.1 The ORM Concept

- **Simple Explanation:** "Object Relational Mapper". It lets you interact with your Database using Python Classes instead of writing SQL strings.
- **Why for you?** Instead of writing `SELECT * FROM users WHERE id=1`, you write `db.query(User).get(1)`. It's cleaner and safer.
- **Keywords to Study:**
  - `Declarative Base`
  - `Models` (Python classes that look like DB tables)
  - `Session` (The temporary connection to the DB)
  - `Filtering` (`.filter()`, `.filter_by()`)

2.3.2 Alembic (Time Travel for DBs)

- **Simple Explanation:** A version control system for your database structure.
- **Why for you?** Today your `Question` table has 3 columns. Tomorrow you want to add `explanation`. Alembic lets you add it safely without deleting all existing questions.
- **Keywords to Study:**
  - `Database Migrations`
  - `alembic init`
  - `alembic revision --autogenerate`
  - `alembic upgrade head`

Phase 2.4: Security & Authentication (The Guard) 🌓

Protecting your data. Trust no one.

2.4.1 Hashing & Salting

- **Simple Explanation:** Never save a password as "password123". Save it as "akjshd872364872".
- **Keywords to Study:**
  - `Encryption` vs `Hashing` (Hashing is one-way)
  - `Bcrypt` algorithm
  - `Salt` (Random data added to passwords to make them unique)

2.4.2 JWT (The VIP Pass)

- **Simple Explanation:** When a user logs in, you give them a digital badge (Token). For every future request, they show the badge to prove who they are. You don't need to check the database for their password again.
- **Keywords to Study:**
  - `JWT` (JSON Web Token) Structure (Header, Payload, Signature)
  - `Bearer Token`

- `Token Expiration`
- `Middleware` (Code that runs before every request to check the token)

Phase 2.5: High-Performance Caching (Redis) 🚀

The Speed. Making your API feel instant.

2.5.1 The Caching Strategy

- **Simple Explanation:** Reading from a Database (Hard Drive) is slow (50ms). Reading from Cache (RAM) is instant (2ms).
- **Why for you?** 10,000 students will ask for the "Physics Syllabus" page. Don't ask the database 10,000 times. Ask once, save it in Redis, and serve the other 9,999 from RAM.
- **Keywords to Study:**
  - `In-Memory Database`
  - `Key-Value Store`
  - `Cache Hit` vs `Cache Miss`
  - `TTL` (Time To Live - How long data stays in cache)
  - `Redis Data Types` (Strings, Lists, Sets)

🛠️ The Capstone Project: "The Secure Backend System"

Goal: Build the engine that powers NEETPrepGPT.

Step 1: Design the DB (SQL Phase)

- Draw the diagram on paper.
- Create tables for `Users`, `Subjects`, `Questions`.
- Use **PostgreSQL** to create them.

Step 2: Build the API (FastAPI Phase)

- Create a `main.py`.
- Build a route `GET /questions` that reads from the DB using **SQLAlchemy**.
- Build a route `POST /questions` that adds a new question (validating it with **Pydantic**).

Step 3: Secure It (Auth Phase)

- Add a `POST /login` route that returns a **JWT**.
- Lock the `POST /questions` route so only users with a valid Token can use it.

Step 4: Speed It Up (Redis Phase)

- Modify `GET /questions`.
- **Logic:** Check Redis -> If empty, Check DB -> Save to Redis -> Return Data.

Module 3: The AI Intelligence Layer

"The Brain, The Memory, and The Reasoning"

Objective: To integrate Large Language Models (LLMs) into your backend. You will learn to make the AI "read" your scraped data and answer student questions accurately without "hallucinating" (lying).

Total Estimated Time: 4–5 Weeks

☑ Definition of Done (Your Checklist)

Do not launch until you can tick these off:

1. ☐ I have an OpenAI API Key configured securely (not hardcoded in the file).
2. ☐ I can explain what an "Embedding Vector" is in plain English.
3. ☐ I have set up a Vector Database (ChromaDB or Pinecone).
4. ☐ I have built a RAG Pipeline: Query -> Search Vector DB -> Send to LLM -> Get Answer.
5. ☐ I can run a test to see if the AI answers a Physics question correctly.

Phase 3.1: LLM Fundamentals (The Brain) 🤖

Understanding the engine before you drive the car.

3.1.1 APIs & Models

- **Simple Explanation:** You don't build the brain; you rent it. You send text to OpenAI, and they send text back.
- **Why for you?** You need to choose the right model. GPT-4 is smart but expensive. GPT-3.5-Turbo (or GPT-4o-mini) is fast and cheap. You need to know when to use which.
- **Keywords to Study:**
- `LLM` (Large Language Model)
- `Tokens` (AI charges by the syllable, not the word. ~0.75 words = 1 token)
- `Temperature` (0.0 = Precise/Robot, 1.0 = Creative/Random)
- `System Prompt` vs `User Prompt`
- `Context Window` (How much text the AI can remember at once)

3.1.2 Prompt Engineering (The Instructions)

- **Simple Explanation:** The AI is a super-smart intern. If you give vague instructions, you get bad results. Prompt Engineering is the skill of writing perfect instructions.
- **Why for you?** You need to tell the AI: "*You are a strict Physics Professor. Only answer based on the provided context. If you don't know, say 'I don't know'.*"
- **Keywords to Study:**
- `Zero-Shot` vs `Few-Shot Prompting` (Giving examples)
- `Chain of Thought` (Asking the AI to "think step-by-step")
- `Prompt Injection` (Security: preventing users from hacking your bot via text)

Phase 3.2: Vector Databases (The Semantic Memory) 🧠

Teaching the AI to "Search by Meaning," not just keywords.

3.2.1 Embeddings

- **Simple Explanation:** Computers don't understand words; they understand numbers. An "Embedding" turns a sentence like "*The mitochondria is the powerhouse of the cell*" into a list of numbers: `[0.12, -0.98, 0.45...]`. Sentences with similar meanings have similar numbers.
- **Why for you?** Standard search looks for *exact words*. Semantic search finds the *concept*. If a student searches "*cell energy factory*", keywords fail. Embeddings know it means "*mitochondria*".
- **Keywords to Study:**
- `Vector Embeddings` (OpenAI `text-embedding-3-small`)
- `Cosine Similarity` (The math to measure how similar two sentences are)
- `High-Dimensional Space`

3.2.2 The Vector Store (ChromaDB / Pinecone)

- **Simple Explanation:** A special database designed to store and search these number lists (vectors) extremely fast.
- **Why for you?** You will convert all your 10,000 scraped MCQs into vectors and store them here.
- **Keywords to Study:**
- `Vector DB`
- `ChromaDB` (Open-source, runs locally - Good for starting)
- `Pinecone` (Managed cloud service - Good for production)
- `Upsert` (Update or Insert)

Phase 3.3: RAG Pipeline (The Engine) ⚙️

Retrieval-Augmented Generation. The Core of NEETPrepGPT.

3.3.1 The Concept

- **Simple Explanation:**

1. **Retrieval:** Student asks a question. You search your Vector DB for the top 3 most relevant textbook paragraphs.
2. **Augmentation:** You paste those paragraphs into the Prompt: "*Use these paragraphs to answer the student's question.*"
3. **Generation:** The AI writes the answer using *your* facts, not its random training data.

- **Why for you?** This stops the AI from lying. It forces it to use your scraped NEET material.
- **Keywords to Study:**
- `RAG` (Retrieval-Augmented Generation)
- `Chunking` (Splitting long text into small pieces before embedding)
- `Retrieval Strategy` (Finding the best matches)

3.3.2 Orchestration (LangChain / LlamaIndex)

- **Simple Explanation:** Libraries that act as "Glue" code to connect OpenAI, your Vector DB, and your Python functions.
- **Keywords to Study:**
- `LangChain` (The most popular framework)
- `LlamaIndex` (Specialized for data indexing)
- `Chains` (Linking steps together)

Phase 3.4: Evaluation & Cost Control

Keeping it accurate and cheap.

3.4.1 Evaluation (The Exam)

- **Simple Explanation:** How do you know if your bot is good? You can't just look at it. You need a "Golden Set" of questions and answers to test it against.
- **Keywords to Study:**
- `RAGAS` (A framework for evaluating RAG pipelines)
- `Hallucination Rate`
- `Ground Truth`

3.4.2 Cost Management

- **Simple Explanation:** Every API call costs money. If you aren't careful, you will go broke.
- **Strategies:**
- **Cache Responses:** If someone asks the same question twice, serve the saved answer (Redis).
- **Rate Limiting:** Don't let one user ask 100 questions in a minute.
- **Token Limits:** Set a `max_tokens` limit on your responses.

🛠️ The Capstone Project: "The AI Tutor Bot"

Goal: Build a script that can answer NEET Biology questions using your scraped data.

Step 1: Ingest (The Setup)

- Load your `clean_data.csv` (from Mod 1).
- Use OpenAI's `text-embedding-3-small` to convert the "Question Text" into Vectors.
- Store these vectors in **ChromaDB**.

Step 2: Search (The Retrieval)

- Write a function `search_db(query)` that converts the user's question to a vector and finds the top 3 most similar MCQs from ChromaDB.

Step 3: Answer (The Generation)

- Write a prompt:

> "You are a NEET Biology Tutor. Use the following Context Questions to explain the answer to the user. Context: {retrieved_mcqs} User Question: {user_query}"

- Send this to **GPT-4o-mini**.

Step 4: The Interface

- Create a simple CLI (Command Line Interface) where you can type a question and get an answer.
- *Input:* "Explain Photosynthesis."
- *Output:* (AI explains using the scraped MCQs as reference).

Module 4: Interfaces & Deployment

"The Ship, The Container, and The Cloud"

Objective: To take your NEETPrepGPT logic and wrap it in a Telegram Bot so students can talk to it. Then, package the whole system in Docker and launch it on a real server.

Total Estimated Time: 3–4 Weeks

☑ Definition of Done (Your Checklist)

Do not call yourself a "Full Stack Developer" until you can tick these off:

1. ☐ I have a Telegram Bot on my phone that answers my questions.
2. ☐ I can explain why "It works on my machine" is a bad excuse and how Docker fixes it.
3. ☐ I have written a `Dockerfile` and can run my app inside a container.
4. ☐ I have a GitHub Actions pipeline that automatically tests my code when I push.
5. ☐ My API and Bot are live on a public URL (like Railway, Render, or AWS).

Phase 4.1: The Interface (Telegram Bot) 🤖

The easiest way to put your AI in user's hands.

4.1.1 Why Telegram?

- **Simple Explanation:** Building a React/Next.js website takes weeks. Building a Telegram Bot takes hours. Users already have the app. It handles notifications, payments, and chat UI for you.
- **Why for you?** Your "MVP" (Minimum Viable Product) needs to launch fast. This is the shortcut.
- **Keywords to Study:**
- `python-telegram-bot` (The library)
- `Polling` (The bot constantly asks "Any new messages?")
- `Webhooks` (Telegram notifies your server "New message!")
- `Handlers` (`CommandHandler`, `MessageHandler`)
- `Context` (Storing user data during a conversation)

4.1.2 Bot Logic Flow

- **Simple Explanation:**

1. User types `/start`. -> Bot says "Welcome! Select a subject."
2. User types "Physics". -> Bot hits your **FastAPI Backend** (Module 2).
3. Backend gets a question from DB.
4. Bot sends the question to User.
5. User answers "Option A". -> Bot checks answer.

- **Keywords to Study:**
- `Finite State Machine` (Managing the flow of conversation)
- `Async Bot` (Handling 1000 users chatting at once)

Phase 4.2: Containerization (Docker)

The "Shipping Container" for code.

4.2.1 The Concept

- **Simple Explanation:** Code is fragile. It runs on your laptop because you have specific versions of Python and Postgres installed. It will crash on a server.
- **The Fix: Docker.** You put your Code, Python, Postgres, and everything it needs into a "Box" (Container). You ship the Box. The Box runs exactly the same everywhere.
- **Why for you?** If you want to be hired or freelance, Docker is mandatory.
- **Keywords to Study:**
- `Image` (The Blueprint / The ISO)
- `Container` (The running instance)
- `Dockerfile` (The recipe to build the image)
- `docker build` / `docker run`
- `Port Mapping` (`-p 8000:8000`)

4.2.2 Docker Compose (Orchestration)

- **Simple Explanation:** Your app has moving parts: The API, The Database, The Redis Cache. Starting them one by one is annoying. `docker-compose` lets you start them all with one command.
- **Keywords to Study:**
- `docker-compose.yml`
- `Services`
- `Volumes` (Persisting DB data so it doesn't vanish when container restarts)
- `Networking` (How the API talks to the DB inside Docker)

Phase 4.3: Deployment (The Cloud)

Going Live.

4.3.1 PaaS (Platform as a Service)

- **Simple Explanation:** The "Easy Mode" of deployment. You just connect your GitHub repo, and they handle the servers.
- **Why for you?** Start here. It's fast and often free for small projects.
- **Keywords to Study:**

- `Render` / `Railway` / `Fly.io`
- `Environment Variables` (Where you hide your secrets like `DB_PASSWORD` in the cloud)
- `Health Checks`

4.3.2 Production Database

- **Simple Explanation:** You can't use a local database on the cloud. You need a hosted PostgreSQL.
- **Keywords to Study:**
- `Managed Database`
- `Connection String` (`postgresql://user:pass@host:port/db`)
- `Database Backups`

Phase 4.4: CI/CD (The Automation) ⚙️

Continuous Integration / Continuous Deployment.

4.4.1 The Pipeline

- **Simple Explanation:**
- **Manual Way:** You change code -> Run tests locally -> SSH into server -> Pull code -> Restart server. (Slow, risky).
- **CI/CD Way:** You push code to GitHub. -> GitHub automatically runs tests. -> If tests pass, GitHub automatically updates the server.
- **Why for you?** This allows you to fix bugs and push updates 10 times a day without breaking a sweat.
- **Keywords to Study:**
- `GitHub Actions`
- `Workflows` (`.yml` files)
- `Jobs` and `Steps`
- `Secrets Management`

🛠️ The Capstone Project: "NEETPrepGPT Live"

The Mission: Launch your product.

Step 1: The Dockerfile

- Write a `Dockerfile` for your FastAPI app.
- *Check:* Can you run `docker run my-app` and see it working?

Step 2: The Bot

- Write a script `bot.py`.
- Connect it to your API.
- Command `/quiz` -> Fetches a random MCQ from your API -> Shows buttons for A/B/C/D.
- User clicks 'A' -> Bot checks API -> Says "Correct! ☑".

Step 3: The Deployment

- Push your code to GitHub.
- Create an account on **Railway.app** (or Render).
- Connect your Repo.
- Add your Environment Variables (`OPENAI_KEY`, `DB_URL`, `TELEGRAM_TOKEN`).
- **Deploy.**

Step 4: The Final Test

- Disconnect your phone from WiFi (use 4G).
- Open Telegram.
- Talk to your bot.
- **If it replies, you have become a Full Stack AI Engineer.**